

AutoHeal-ML: Technical Architecture

Autonomous Multi-Agent API Orchestrator & Self-Healing Data Pipeline

Target Experience:	3+ Years (Mid/Senior Level)	Domains Covered:	Data Analytics, Data Science, AI/ML Engineering
Project Duration:	8 Weeks (4 Sprints)	Key Technologies:	PostgreSQL, XGBoost, SHAP, LangGraph, MCP, FastAPI

1. Executive Abstract

Modern enterprise data pipelines and ML microservices frequently fail due to unannounced API schema breaking changes, data type mismatches, and data drift. Resolving these issues manually consumes hundreds of engineering hours, leading to prolonged Mean Time to Resolution (MTTR).

AutoHeal-ML is a production-grade, self-healing MLOps and Data Engineering platform. The system continuously ingests telemetry streams, utilizing advanced PostgreSQL window functions to detect anomaly spikes in real time. Upon failure detection, an XGBoost root-cause classifier paired with SHAP (SHapley Additive exPlanations) isolates the exact breaking feature or payload schema change.

A LangGraph multi-agent architecture integrated via Model Context Protocol (MCP) then inspects the application codebase, writes a backward-compatible code patch, validates it against unit tests within a sandboxed Docker container, and subjects it to NeMo Guardrails and RAGAS evaluation suites. Upon validation, the platform automatically generates a GitHub Pull Request while streaming real-time diagnostic telemetry to a FastAPI/WebSocket Streamlit dashboard, tracked end-to-end via MLflow.

2. Architectural Core Pillars

* Data Analytics Pillar (PostgreSQL & SQL Engine)

Handles stream telemetry storage, payload execution logs, and window function transformations. Computes rolling statistical Z-scores ($Z > 3.0$) over sliding time windows to detect performance anomalies instantly.

* Data Science Pillar (Predictive ML & SHAP Explainability)

Trains XGBoost models to classify failure modes and uses SHAP to calculate exact feature importance values. Eliminates black-box predictions by identifying the specific key or metric triggering the failure.

* AI / ML Engineering Pillar (LangGraph, MCP & RAG)

Deploys multi-agent orchestration using Model Context Protocol (MCP) tools to navigate codebases, inspect broken scripts, generate patches, and run sandboxed tests evaluated via NeMo Guardrails and RAGAS.

* MLOps & Production Engineering Pillar (Docker, FastAPI & MLflow)

Exposes REST/WebSockets endpoints, tracks experiment runs and prompt versions in MLflow, containerizes microservices via Docker Compose, and automates GitHub Pull Requests.

3. Detailed 4-Sprint Implementation Plan

Sprint 1: Data Analytics & SQL Anomaly Engine (Weeks 1 - 2)

- Define PostgreSQL schema for telemetry, execution latencies, and response codes.
- Build a synthetic data generator mimicking payload schema breaks and latency spikes.
- Write complex SQL window functions & CTEs calculating rolling Z-score metrics.
- Deliverable: Running PostgreSQL database flagging payload anomalies in real time.

Sprint 2: ML Root-Cause & Explainability Engine (Weeks 3 - 4)

- Train XGBoost / Isolation Forest model to classify root cause failure types.
- Integrate SHAP library to output local feature-level attributions for alerts.
- Implement Evidently AI pipelines to track feature and concept drift over time.
- Deliverable: Diagnostic payload output (e.g., Failure: SchemaBreak | Top Feature: missing_key_user_tier).

Sprint 3: Agentic GenAI, MCP & Evaluation Pipeline (Weeks 5 - 6)

- Construct custom Model Context Protocol (MCP) server tools for repository access.
- Build LangGraph state machine with Investigator, Developer, and Validator agents.
- Apply NeMo Guardrails for security and evaluate prompt metrics using RAGAS.
- Deliverable: Autonomous agent script generating passing, unit-tested code patches.

Sprint 4: Production MLOps, UI & Cloud Deployment (Weeks 7 - 8)

- Build FastAPI REST/WebSockets backend for live streaming diagnostic traces.
- Construct Streamlit dashboard with Z-score charts, SHAP plots, and side-by-side git diffs.
- Containerize the entire stack using Docker Compose and integrate MLflow tracking.
- Deliverable: Live, deployable system capable of auto-generating GitHub PRs.